MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Vector Databases & RAG

Finding the right five paragraphs in a million — and answering from them honestly.

Unit 06 · Part 2 of 2 · Session 09 · Week 9
Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Vectors & similarity search** embeddings, ANN indexes, HNSW
2. **The vector DB landscape** four categories, not forty products
3. **Chunking & the RAG pipeline** ingest, retrieve, ground the answer
4. **Better retrieval** hybrid search, reranking, agentic RAG
5. **Evaluation & limits** retrieval metrics, faithfulness, authorization, Lab 09

Where we are

Unit 06, part 2 — from what to remember to how to find it

Last week (S08) — memory taxonomy: what to store, when to write, when to forget

This week — the machinery underneath: storing and searching at scale

The link — similarity search is one retrieval primitive, combined with metadata filters, structured queries and access control

Ahead (S10) — agents acting on retrieved, untrusted content — safety becomes the topic

GUIDING QUESTION

How does an agent find the five relevant paragraphs among a million it has stored — fast, and without lying about them?

1

Vectors and how to search them

Embeddings, approximate nearest neighbours, HNSW

Embeddings: meaning as geometry

A brief recap — you know the transformer underneath

Embedding — a learned map from a text span to one dense vector (often hundreds to a few thousand dims)

Geometry as meaning — trained so paraphrases land close, unrelated texts land far apart

Cosine similarity — compares direction, not length — the standard relevance score

Contrastive training — pull related pairs together, push others apart (Reimers & Gurevych, 2019)

Retrieval asymmetry — questions do not look like answers — use retrieval-trained models

YOU KNOW THIS PART

An encoder pass, pooled to one vector per text — not per token. Today we treat the embedding model as a primitive.

HNSW: the workhorse ANN index

Brute force first, then Hierarchical Navigable Small World graphs (Malkov & Yashunin, 2018)

Exact baseline — a brute-force scan costs O(N·d) per query: fine at thousands of vectors, not at hundreds of millions

Structure — a layered proximity graph: sparse long-range links on top, dense local links below

Search — greedy descent: enter at the top, hop closer, drop a layer, refine

Complexity — approximately logarithmic search behaviour in practice; no general worst-case guarantee

Parameters — M, efConstruction, efSearch: recall, memory and latency depend on data and settings

Updates — inserts usually cheap, deletes often tombstoned; RAM residency is implementation-specific

THE TRADE

ANN is a tunable, empirical recall-latency trade-off. Measure recall on your own data instead of quoting a generic figure.

The vector database landscape

Deployment categories as examples, not exclusive technical properties

Category	Examples	Deployment	Choose when
Library	FAISS	In-process index, no server	Research, batch jobs, full control
Embedded DB	Chroma, LanceDB	In-process, persists to disk	Prototypes, local agents, single node
SQL extension	pgvector	Inside your existing Postgres	Vectors beside relational data, one system
Dedicated store	Qdrant, Weaviate, Milvus, Pinecone	Separate service or managed cloud	Scale, filtering, replication, multi-tenancy

Operational differences dominate the choice, but index types differ too: flat, IVF/PQ, HNSW, disk-based.

2

Chunking and the RAG pipeline

How you split decides what can be found

Chunking strategies

One vector must summarise each chunk — split accordingly

Fixed-size windows

Split every N tokens, with overlap. Simple, fast, format-agnostic, but cuts sentences and tables. Small chunks: sharp vectors, dangling context; large: self-contained, blurry, prompt-hungry.

Structural

Split at headings, paragraphs, code blocks, table rows. Respects the author's logic; needs format-aware parsing per document type.

Semantic

Split where embedding similarity between adjacent sentences drops. Topically coherent chunks; costs an extra embedding pass.

Contextual enrichment

Use the full document to generate a short chunk-specific context, prepend it to the chunk, then index it densely and lexically (Anthropic, 2024).

The classic RAG pipeline

A common RAG shape today, reranker optional (after Lewis et al., 2020)



- Ingest and index run once per document; retrieve, rerank, generate run per query.
- The generator prompt: instructions, retrieved chunks with IDs, then the question.
- This is single-shot — one retrieval, one answer. The agentic upgrade comes in section 4.

Grounding the generation

The prompt requests source-grounded answers; claim-evidence checks assess compliance

Prompt assembly — system rules, then retrieved chunks with IDs, then the question

Core instruction — answer only from the provided sources; cite a chunk ID per claim

Refusal path — if the evidence is missing, say so — never improvise

Position effects — models privilege the start and end of long contexts (Liu et al., 2024) — weaker in 2026 models, still measurable

Citations — chunk ID, document version and evidence span — a bare URL drifts when the page changes

LAB 09, PART D

System rules, chunks with IDs, question. A fixed refusal sentence is tested on an unanswerable query: fluent but ungrounded answers are the failure to catch.

3

Better retrieval

Hybrid search and reranking

Dense and lexical retrieval fail differently

Embeddings vs BM25 (Robertson & Zaragoza, 2009)

Dense (embeddings)

- matches meaning: paraphrase, synonyms
- robust to vocabulary mismatch
- weak on exact identifiers and codes
- opaque: hard to explain a hit

Lexical (BM25)

- matches exact terms, weighted by rarity
- wins on IDs, error codes, names, versions
- no training, no embedding cost, explainable
- blind to synonyms and paraphrase

Hybrid retrieval and reranking

Two cheap recalls, one expensive precision pass

Hybrid retrieval — run BM25 and dense search in parallel; union the candidate lists

Rank fusion — RRF merges by rank, a robust baseline (Cormack et al., 2009); calibrated score fusion is an option

Cross-encoder rerank — scores each query-chunk pair jointly, usually batched; accurate, too costly for millions

Two-stage shape — cheap recall over millions of chunks, expensive precision over the top fifty

WHY RERANK AT ALL?

A bi-encoder compresses each text to one vector before seeing the query. The cross-encoder sees both together — it just cannot afford to see everything.

4

RAG in the agent

Against the alternatives — and as a tool in the loop

RAG vs long context vs fine-tuning

Three ways to get knowledge into a model

Axis	RAG	Long context	Fine-tuning
Freshness	re-index changed documents, small lag	re-send documents per call	frozen at training time
Cost per query	depends on corpus, cache, retrieval pipeline	depends on tokens sent and caching	low inference, high upfront
Provenance	citations to retrieved chunks	possible, harder to pin	none — knowledge is implicit
Corpus size	millions of documents	bounded by the window	bounded by training budget
Access control	enforced per user in the retrieval service	caller selects the documents	none per user — enforce externally
Best for	facts, changing knowledge	small corpora, dense cross-references	style, format, behaviour

No general cost or latency ranking: compare for your corpus, model and cache. Production systems combine RAG for facts with fine-tuning for behaviour.

Agentic RAG: retrieval as a tool

From a fixed pipeline to a loop the agent controls (Lab 09, Part F)



- Multi-hop: the answer to hop one becomes the query for hop two.
- Self-correcting: critique the evidence, re-retrieve on weak support (Asai et al., 2023).
- Each loop costs tokens and latency: Lab 09 assesses the best similarity, rewrites once, keeps the better hits.

Retrieval as a tool (1): ingest into Chroma

Embedded store, explicit local embedding model and cosine distance; stable chunk ids and versions for citations

retrieval_tool.py (part 1)

```
import chromadb.utils.embedding_functions as ef
from chromadb import PersistentClient
emb = ef.OllamaEmbeddingFunction(
    model_name="nomic-embed-text")  # same model at query time
client = PersistentClient("agent_memory")
col = client.get_or_create_collection(
    "sources", embedding_function=emb,
    metadata={"hnsw:space": "cosine"})
if (col.metadata or {}).get("hnsw:space") != "cosine":
    raise RuntimeError("index not cosine: re-index")
def ingest(chunks):  # id, text, url, version, tenant
    meta = [{k: c[k] for k in ("url", "version", "tenant")}
            for c in chunks]
    col.upsert(ids=[c["id"] for c in chunks],
               documents=[c["text"] for c in chunks],
               metadatas=meta)
```

Retrieval as a tool (2): query with ids and distances

Tenant authorization comes from the runtime's caller, never a model-chosen filter; dist is a distance, not a confidence

retrieval_tool.py (part 2)

```
def search_sources(query, caller, k=5) -> list[dict]:
    """Top-k passages; dist = cosine distance (0 = same)."""
    # tenant filter set by the runtime, not a model argument
    res = col.query(query_texts=[query], n_results=k,
                    where={"tenant": caller.tenant})
    if not res["ids"] or not res["ids"][0]:
        return []  # no evidence: the agent must say so
    out = []
    for i, d, m, s in zip(res["ids"][0], res["documents"][0],
                          res["metadatas"][0],
                          res["distances"][0]):
        m = m or {}  # metadata may be missing
        out.append({"id": i, "text": d, "url": m.get("url"),
                    "version": m.get("version"), "dist": s})
    return out
# Registered as a tool; citations carry id and version.
```

5

Evaluation and limits

Measuring retrieval, judging faithfulness, honest claims

Evaluating retrieval first

Retrieval quality strongly shapes grounded answering, but it is not a numerical ceiling

Ground truth — a small, curated set of queries labelled with their relevant chunk IDs

Recall@k / hit rate@k — share of relevant chunks (or of queries with a hit) in the top k

Precision@k — share of the top k that is actually relevant — prompt hygiene

MRR / nDCG — rank-aware scores: reward putting the right chunk first

Not a hard bound — one sufficient chunk can answer at recall 0.1; also measure evidence sufficiency and answer quality

Low recall? — fix chunking, embeddings, query rewriting or multi-step retrieval before blaming the generator

LAB 09, PART E

Hit rate@5 over 13 labelled queries, then MRR@5 (R3), before any generated answer is judged. A chunk that never reaches the prompt cannot be cited.

Evaluating the answer and the system

Six distinct questions — do not collapse them into one score

Faithfulness

Is every claim in the answer entailed by the retrieved chunks? Checked claim by claim, usually with an LLM judge (Es et al., 2023).

Answer relevance

Does the answer address the question that was asked, rather than summarising whatever happened to be retrieved?

Citation accuracy

Does each citation resolve to a chunk ID and document version that supports its sentence? Judges miss subtle mismatches: spot-check by hand.

Evidence sufficiency

Did the retrieved set contain enough to answer at all? Judged per query, independent of chunk recall.

Authorization tests

Can a user ever receive another tenant's chunks, or deleted and revoked documents, via retrieval, reranking, caches or citations?

Faithful is not correct

A faithful answer is still wrong when the corpus is wrong. RAG shifts trust from the model to the corpus.

Hallucination: reduced, not solved

What grounding buys — and what it cannot

What RAG reduces — guessing about facts the corpus covers, when retrieval hits and the model complies

Silent blending — models mix retrieved facts with parametric memory without marking which is which

Retrieval failure — empty or off-topic context invites confident improvisation

Conflicting sources — the model tends to pick one silently instead of flagging the conflict

The honest claim — RAG lowers hallucination rates and makes errors auditable — it does not eliminate them

CITATIONS HELP TWICE

They discipline generation and they let a human verify. Auditable wrongness beats plausible opacity.

Lab 09: a plain-NumPy RAG pipeline

Gap-fill notebook, local Ollama (qwen2.5:7b, nomic-embed-text), offline corpus, Parts A-G, 90-120 min, report tasks R1-R4

B: Corpus and chunking

Six short synthetic handbook documents (data/*.md), all facts invented so correct answers must come from retrieval. Fixed-size word windows with overlap.

C: Embed, index, retrieve

nomic-embed-text with search_document and search_query prefixes; the index is one NumPy matrix with an exact cosine scan; retrieve returns the scores.

D: Grounded generation

Prompt: rules, chunks with IDs, question. Cite a chunk ID per claim; a fixed refusal sentence, tested on an unanswerable query (R2).

E: Evaluate retrieval first

13 labelled queries, 10 answerable: hit rate@k, then implement MRR@5 (R3), before judging any generated answer.

F: Assess and re-retrieve

If the best similarity is below a threshold, the LLM rewrites the query, retrieves once more and keeps the better hit set (R4).

G: Tuning sweep

Chunk size 40-320 words, overlap and k against hit rate; inspect what a hit feeds the generator; recommend an operating point (R1).

TAKEAWAYS

What you should remember

- **Embeddings —** meaning as geometry — cosine similarity finds paraphrase, not keywords
- **ANN / HNSW —** approximately logarithmic in practice; recall, memory, latency depend on data and parameters
- **Landscape —** choose by deployment category: library, embedded, Postgres, dedicated
- **Pipeline —** chunk, embed, index — retrieve, optionally rerank, generate a grounded, cited answer
- **Agentic RAG —** retrieval as a tool: rewrite, multi-hop, assess, stop when evidence suffices
- **Evaluation —** measure retrieval first, but chunk recall is no hard ceiling; faithfulness is judged, never assumed

NEXT SESSION Safe operation — sandboxing and guardrails.

Reading & references

- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401.
  The paper that named the pattern.
- Malkov, Y., Yashunin, D. (2018). Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs. IEEE TPAMI. arXiv:1603.09320.
- Johnson, J., Douze, M., Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. IEEE Trans. on Big Data. arXiv:1702.08734.
  The FAISS paper.
- Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. arXiv:1908.10084.
- Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR.
- Cormack, G., Clarke, C., Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.
- Asai, A. et al. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511.
- Es, S. et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. EACL 2024 demo; arXiv:2309.15217.
  Faithfulness, answer relevance, context precision/recall.
- Anthropic (2024). Introducing Contextual Retrieval. Engineering blog, September 2024.